

СОДЕРЖАНИЕ

Введение	3
1 Структура и краткая характеристика предприятия	5
1.1 Информация о компании	5
1.2 Структура предприятия	5
1.3 Структура подразделения производственной практики	6
2 Анализ предметной области	7
2.1 Теоретические основы разработки	7
2.2 Обзор программных аналогов	8
2.3 Постановка задачи разработки	8
2.4 Анализ архитектуры платформы Century	9
3 Обзор используемых технологий и средств разработки	11
3.1 Технологический стек проекта	11
3.2 Язык и библиотеки	12
4 Проектирование и разработка программного модуля	13
4.1 Функциональность сервиса	13
4.2 Декомпозиция на модули	13
4.3 Контроль нагрузки	16
4.4 Инфраструктура наблюдаемости	17
4.5 Доработка микросервисов платформы Century	18
5 Тестирование разработанного решения	20
5.1 Верификация архитектуры	20
5.2 Валидация системы очередей	20
5.3 Проверка отказоустойчивости	22
5.4 Тестирование доработок микросервисов платформы	22
Заключение	25
Список литературных источников	26
Приложение А (обязательное) Листинг программного кода	27

ВВЕДЕНИЕ

Практика проходила на предприятии ООО «СТЭКЛЭВЭЛ ГРУПП» (г. Минск). В ходе практики была выполнена разработка и доработка компонентов корпоративной AI-платформы Century, а также HTTP-сервера GigaServer для обработки аудиозаписей с использованием технологий диаризации спикеров и автоматического распознавания речи.

Система GigaServer предназначена для автоматической обработки аудиофайлов: определения спикеров на всей записи с помощью модели NVIDIA Sortformer и распознавания речи каждой реплики моделью GigaAM v3. Сервис предоставляет OpenAI-совместимый REST API и поддерживает работу как на CPU, так и на GPU с автоматическим переключением при сбоях.

Платформа Century представляет собой корпоративное решение на базе микросервисной архитектуры для управления AI-агентами, обработки диалогов и интеграции с языковыми моделями. В рамках производственной практики выполнялись задачи по анализу и доработке ключевых микросервисов платформы: управления агентами, истории сообщений, координации мультиагентов.

Основные цели производственной практики:

- 1 Изучить архитектуру микросервисной AI-платформы Century и принципы взаимодействия её компонентов.

- 2 Освоить методы разработки и сопровождения распределённых систем на базе FastAPI и PostgreSQL.

- 3 Спроектировать и реализовать модульную архитектуру HTTP-сервера для обработки аудио с интеграцией ML-моделей.

- 4 Разработать систему управления очередями запросов и механизмы контроля нагрузки на сервис.

- 5 Реализовать комплексную систему логирования и мониторинга для production-окружения.

- 6 Провести тестирование разработанных решений и подтвердить их работоспособность.

Задачи производственной практики:

- 1 Изучение архитектуры платформы Century: анализ микросервисов управления агентами, истории сообщений, мультиагентов; выявление паттернов взаимодействия компонентов через REST API.

- 2 Доработка микросервисов платформы: оптимизация запросов к базе данных, улучшение обработки ошибок, расширение функциональности API, повышение поддерживаемости кода.

- 3 Рефакторинг HTTP-сервера GigaServer: разделение монолитного файла на специализированные модули с чёткой ответственностью, вынесение конфигурации, логики работы с аудио, управления моделями в отдельные компоненты.

- 4 Разработка системы управления очередями: реализация двухуровневой системы контроля нагрузки (асинхронная очередь HTTP-запросов и синхронная очередь инференса), поддержка различных режимов

ожидания и корректная координация между уровнями.

5 Реализация системы логирования: настройка структурированных логов с ротацией файлов, внедрение сквозной трассировки запросов через идентификаторы, добавление метрик производительности для мониторинга.

6 Разработка механизма отказоустойчивости: реализация автоматического переключения с GPU на CPU при ошибках, обеспечение стабильной работы сервиса в нестабильной среде.

7 Тестирование разработанных компонентов: модульное и интеграционное тестирование, проверка работы под нагрузкой, валидация API-контрактов.

Работа над практикой включала несколько этапов: ознакомление с деятельностью предприятия и изучение архитектуры платформы Century, анализ существующего кода микросервисов и HTTP-сервера с выявлением направлений для улучшения, реализацию оптимизаций запросов к базе данных и расширение функциональности API микросервисов, проектирование модульной архитектуры GigaServer и реализацию базовых компонентов (конфигурация, логирование, разделение транспорта и бизнес-логики), разработку системы управления очередями запросов и механизма автоматического fallback с CUDA на CPU, интеграцию метрик производительности и системы трассировки, тестирование на локальном стенде и в production-окружении, исправление выявленных ошибок и оптимизацию производительности, подготовку итоговой документации.

1 СТРУКТУРА И КРАТКАЯ ХАРАКТЕРИСТИКА ПРЕДПРИЯТИЯ

1.1 Информация о компании

Общество с ограниченной ответственностью «СТЭКЛЭВЭЛ ГРУПП» – белорусская IT-компания, работающая в г. Минске [1]. Предприятие специализируется на разработке корпоративных решений в области искусственного интеллекта, создании платформ для работы с большими языковыми моделями и автоматизации бизнес-процессов с использованием современных технологий машинного обучения. Основные направления деятельности предприятия:

- разработка корпоративных AI-платформ для управления интеллектуальными агентами и обработки естественного языка;
- создание систем автоматического распознавания речи и обработки аудиоданных для корпоративных клиентов;
- интеграция больших языковых моделей в бизнес-процессы организаций;
- разработка микросервисных архитектур и распределённых систем;
- консалтинг и техническая поддержка внедрённых решений.

Основные проблемы и задачи, решаемые предприятием:

- создание масштабируемых систем для обработки больших объёмов данных с использованием AI-технологий;
- обеспечение высокой доступности и отказоустойчивости критичных сервисов;
- оптимизация использования вычислительных ресурсов (GPU/CPU) для ML-моделей;
- автоматизация обработки аудиозаписей разговоров с выделением спикеров и транскрипцией;
- разработка удобных API для интеграции AI-функциональности в существующие системы клиентов.

1.2 Структура предприятия

ООО «СТЭКЛЭВЭЛ ГРУПП» имеет компактную организационную структуру. Во главе предприятия находится генеральный директор, который организует деятельность подразделений и несёт ответственность за стратегическое развитие компании и взаимодействие с клиентами.

Среди ключевых функциональных направлений предприятия:

- отдел разработки (проектирование архитектуры решений, разработка backend-сервисов на Python, интеграция ML-моделей, создание API);
- отдел машинного обучения (подбор и адаптация моделей распознавания речи и обработки языка, оптимизация производительности инференса);

- отдел DevOps и инфраструктуры (развёртывание сервисов, мониторинг производительности, обеспечение отказоустойчивости);
- отдел продукта и аналитики (сбор требований от клиентов, проектирование функциональности, приёмочное тестирование);
- административно-хозяйственные службы (бухгалтерия, кадры, юридическое сопровождение).

Предприятие работает с корпоративными клиентами из различных отраслей, предоставляя как готовые решения, так и услуги по кастомной разработке под специфические требования. Технологический стек компании включает современные инструменты: Python, FastAPI, PostgreSQL, Docker, PyTorch, различные ML-фреймворки для работы с аудио и текстом.

1.3 Структура подразделения производственной практики

Производственная практика проходила в подразделении разработки backend-сервисов, ответственном за создание и сопровождение серверных компонентов AI-платформы Century и специализированных сервисов обработки данных. Данное направление является ключевым для деятельности предприятия, поскольку через него обеспечивается техническая реализация всех продуктов компании.

Основные задачи подразделения:

- 1 Проектирование и разработка микросервисных архитектур для AI-платформ с учётом требований масштабируемости и отказоустойчивости.
- 2 Разработка REST API для взаимодействия между компонентами распределённых систем и для интеграции с клиентскими приложениями.
- 3 Интеграция ML-моделей в production-сервисы с обеспечением требуемой производительности и надёжности.
- 4 Оптимизация использования вычислительных ресурсов, реализация механизмов управления очередями и контроля нагрузки.
- 5 Разработка систем логирования, мониторинга и трассировки для упрощения отладки и поддержки сервисов в production.
- 6 Проведение code review, рефакторинга существующего кода и внедрение best practices разработки.

Место подразделения в структуре предприятия – центральное: его сотрудники взаимодействуют с отделом машинного обучения для интеграции моделей, с DevOps для развёртывания сервисов, с продуктовой командой для уточнения требований. Работа подразделения напрямую влияет на качество и стабильность предоставляемых клиентам решений.

В рамках производственной практики основное время было посвящено двум направлениям: работе с микросервисами платформы Century (изучение архитектуры, анализ и доработка компонентов) и рефакторингу HTTP-сервера GigaServer с реализацией модульной архитектуры и систем управления очередями и логирования.

2 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Существующий HTTP-сервер GigaServer функционировал на основе монолитной архитектуры: весь код размещался в едином файле `main.py` размером около 900 строк. Такая организация кода приводила к смешению различных уровней абстракции – конфигурационные параметры, HTTP-обработчики, бизнес-логика работы с ML-моделями, утилиты для конвертации аудио и функции форматирования результатов находились в одном файловом пространстве. Подобная структура создавала существенные проблемы при поддержке: любое изменение требовало анализа всего файла, тестирование отдельных компонентов было затруднено из-за тесной связанности, а внесение новой функциональности несло риски непредвиденных побочных эффектов.

Другой значимой проблемой являлось отсутствие механизмов контроля нагрузки. При поступлении большого количества одновременных запросов сервис мог принять их все без ограничений, что приводило к исчерпанию оперативной памяти при загрузке ML-моделей и CPU-ресурсов при выполнении инференса. Отсутствие системы очередей делало поведение сервиса непредсказуемым под нагрузкой.

Логирование работы сервиса было реализовано минималистично: простой вывод в `stdout` без структурирования, отсутствие идентификаторов для трассировки отдельных запросов, недостаточная детализация событий. Это существенно усложняло диагностику проблем в production-окружении, где одновременно обрабатываются запросы от множества клиентов.

2.1 Теоретические основы разработки

При проектировании серверных приложений, работающих с ML-моделями, необходимо учитывать специфику организации кода и управления вычислительными ресурсами. Современная практика разработки опирается на принципы модульности и разделения ответственности между компонентами системы.

Ключевым принципом является организация кода по принципу единственной ответственности (Single Responsibility Principle): каждый модуль должен решать строго определённую задачу и иметь одну причину для изменения. Применительно к HTTP-сервису это означает выделение отдельных модулей для конфигурации, транспортного уровня, бизнес-логики и вспомогательных операций. Такая организация упрощает понимание кода, тестирование и внесение изменений.

Управление вычислительными ресурсами в системах с ML-моделями требует применения паттерна пула ресурсов (Resource Pool Pattern). Инференс моделей глубокого обучения потребляет значительные объёмы памяти и процессорного времени, поэтому одновременное выполнение неограниченного числа операций может привести к отказу в обслуживании. Решением является создание пула фиксированного размера и очереди ожидания с контролируруемыми таймаутами.

Для production-систем критична observability – возможность наблюдения за внутренним состоянием системы через её выходные данные. Это достигается через структурированное логирование с едиными форматами записей, распределённую трассировку запросов (distributed tracing), метрики производительности и эндпоинты health check. Совокупность этих механизмов позволяет оперативно обнаруживать и диагностировать проблемы.

Принцип graceful degradation предполагает, что система должна сохранять работоспособность при частичном отказе компонентов. Для сервисов с GPU-ускорением это реализуется через механизм автоматического fallback на CPU: при возникновении ошибок CUDA система переключается на процессорные вычисления, обеспечивая непрерывность обслуживания за счёт снижения производительности.

2.2 Обзор программных аналогов

Анализ рынка показывает наличие нескольких категорий решений для развёртывания ML-моделей в production. Специализированные фреймворки для обработки речи, такие как Kaldi Gstreamer Server, предоставляют готовую инфраструктуру, но ориентированы на конкретные модели семейства Kaldi. Их адаптация под NVIDIA NeMo Sortformer и ONNX-модели GigaAM требует значительных усилий.

Универсальные платформы model serving (TensorFlow Serving, TorchServe, NVIDIA Triton Inference Server) решают задачу развёртывания моделей, но избыточны для узкоспециализированного сервиса. Они добавляют сложность в конфигурацию, требуют изучения специфичных форматов описания моделей и не предоставляют готовых решений для комбинированных пайплайнов «диаризация + ASR».

Разработка кастомного решения на базе веб-фреймворка предоставляет полный контроль над архитектурой и поведением системы. Фреймворки для Python (FastAPI, Flask, aiohttp) позволяют создать HTTP API с асинхронной обработкой и интеграцией произвольных библиотек. При этом разработчик несёт полную ответственность за реализацию механизмов управления ресурсами, очередей, логирования и отказоустойчивости.

Сравнительный анализ показал, что специфика задачи (OpenAI-совместимый API, пайплайн диаризация + ASR, работа с конкретными моделями) делает кастомную разработку наиболее рациональным выбором. FastAPI выбран как основа благодаря встроенной поддержке асинхронности, автоматической генерации документации и активному сообществу.

2.3 Постановка задачи разработки

На основании проведённого анализа были сформулированы технические требования к разработке.

Архитектурный рефакторинг. Необходимо декомпонизировать монолитный файл на независимые модули с явно определёнными

интерфейсами. Предполагается выделение следующих компонентов: модуль конфигурации (чтение переменных окружения, настройка логирования), модуль HTTP-транспорта (обработка запросов, middleware, lifespan-события), модуль управления ML-моделями (загрузка, инференс, fallback), модуль работы с аудио (декодирование форматов, конвертация), модуль форматирования (преобразование результатов в различные форматы ответов), модуль метрик (сбор статистики).

Система управления нагрузкой. Требуется разработать двухуровневую систему контроля: на уровне HTTP реализовать асинхронную очередь через `asyncio.Semaphore` с настраиваемым размером пула и таймаутом ожидания, на уровне инференса реализовать синхронную очередь через `threading.Semaphore` для обеспечения совместимости с блокирующими операциями. Система должна поддерживать три режима: блокирующее ожидание без таймаута, немедленный отказ при отсутствии свободных слотов, ожидание с заданным таймаутом.

Инфраструктура логирования. Необходимо реализовать многоуровневую систему: структурированный формат записей с метаданными (timestamp, log level, process ID, thread name, request ID), запись одновременно в stdout и ротлируемый файл на диске, сквозную трассировку запросов через контекстные переменные, специальный обработчик для критических падений процесса.

Механизм отказоустойчивости. Требуется автоматическое определение CUDA-специфичных ошибок и переключение на CPU-режим. Fallback должен работать на двух этапах: при инициализации моделей (автоматический retry загрузки на CPU при CUDA-ошибке), во время обработки запросов (перезагрузка моделей на CPU при runtime-ошибке). Переключение должно быть координированным: блокировка новых запросов, ожидание завершения активных, замена моделей, возобновление обработки.

Мониторинг состояния. Health check эндпоинт должен возвращать полную информацию: статус готовности моделей, используемое устройство (CPU/GPU), размер пула инференса, количество свободных слотов, агрегированную статистику запросов (total, success, failed с детализацией по стадиям обработки).

2.4 Анализ архитектуры платформы Century

Платформа Century представляет собой корпоративное решение для управления AI-агентами и обработки диалогов на основе больших языковых моделей. Архитектура построена на принципах микросервисного подхода с явным разделением ответственности между компонентами системы.

Основные характеристики архитектуры включают изолированные базы данных PostgreSQL для каждого микросервиса со своей схемой и миграциями, взаимодействие между компонентами через REST API с использованием HTTP-клиентов, разработку всех сервисов на Python с использованием FastAPI или Flask в качестве веб-фреймворка, контейнеризацию через Docker с

оркестрацией через docker-compose, асинхронную обработку задач через очереди сообщений RabbitMQ и файловое хранилище на базе MinIO (S3-совместимое объектное хранилище).

Ключевые микросервисы платформы представлены следующими компонентами. AgentBuilder отвечает за управление конфигурациями AI-агентов, их промптами, параметрами взаимодействия с языковыми моделями, инструментами и возможностями. ConversationService (chat_core_service) обеспечивает хранение истории диалогов пользователей с агентами, управление контекстом беседы, обработку вложений и файлов. BackendService выступает центральным сервисом оркестрации, координирующим взаимодействие между агентами, LLM-провайдерами, векторными базами и другими компонентами. Multi-agent сервис реализует сценарии работы нескольких агентов в рамках одной задачи и координацию их взаимодействия. LLM_service предоставляет абстракцию над различными провайдерами языковых моделей (OpenAI, Anthropic, локальные модели). Search_service и vector_service обеспечивают семантический поиск и работу с векторными представлениями документов. Transcription_service выполняет интеграцию с сервисами распознавания речи для обработки аудиофайлов. File_service управляет загрузкой, хранением и обработкой пользовательских файлов.

В ходе практики был проведён анализ существующих решений с выявлением направлений для улучшения. Основные проблемные области включали недостаточную оптимизацию запросов к базе данных (избыточные N+1 запросы, отсутствие индексов на часто используемых полях), неполную обработку граничных случаев и ошибок (отсутствие валидации входных данных, некорректная обработка таймаутов внешних сервисов), возможности для расширения функциональности API (дополнительные параметры фильтрации, пагинация, сортировка), а также необходимость улучшения читаемости и поддерживаемости кода (рефакторинг дублирующейся логики, вынесение конфигурационных параметров, улучшение структуры модулей).

3 ОБЗОР ИСПОЛЬЗУЕМЫХ ТЕХНОЛОГИЙ И СРЕДСТВ РАЗРАБОТКИ

3.1 Технологический стек проекта

Основой серверной части приложения выбран веб-фреймворк FastAPI для языка Python [2]. Данный фреймворк построен на базе современных возможностей Python 3.6+ и предоставляет встроенную поддержку асинхронного программирования, автоматическую валидацию входных данных через type hints, генерацию интерактивной документации API в формате OpenAPI/Swagger. FastAPI показывает производительность, сопоставимую с Node.js и Go-фреймворками, благодаря использованию Starlette в качестве транспортного слоя и Pydantic для валидации данных.

Работа с моделями машинного обучения осуществляется посредством двух фреймворков. Для диаризации спикеров применяется NVIDIA NeMo – специализированный toolkit для разработки conversational AI приложений [3]. NeMo предоставляет предобученные модели семейства Sortformer, оптимизированные для работы на GPU с поддержкой CUDA. Распознавание речи выполняется моделью GigaAM v3 в формате ONNX с использованием ONNX Runtime – кроссплатформенного движка инференса, обеспечивающего высокую производительность как на CPU, так и на GPU за счёт оптимизированных операторов и graph optimizations [4].

Предварительная обработка аудиофайлов реализована через интеграцию с ffmpeg – мультимедийным фреймворком, поддерживающим декодирование широкого спектра форматов (WAV, OGG, Vorbis, Opus, M4A/AAC, MP4, FLAC). Ffmpeg выполняет ресемплинг до требуемой частоты дискретизации 16 кГц, конвертацию в PCM формат с разрядностью 16 бит и микширование до моноканала при необходимости [5].

В контексте платформы Century для персистентности данных используется объектно-реляционная СУБД PostgreSQL [6]. База данных обеспечивает ACID-гарантии транзакций, поддержку JSON-типов для хранения полуструктурированных данных, механизмы индексирования для оптимизации запросов. Каждый микросервис платформы оперирует собственной схемой данных, что обеспечивает изоляцию и независимость компонентов.

Контейнеризация приложения выполнена средствами Docker [7]. Образ сервиса включает runtime-зависимости (Python, системные библиотеки, ffmpeg), ML-библиотеки (PyTorch, NeMo, ONNX Runtime) и предварительно скачанные веса моделей для ускорения первого запуска. Образ поддерживает универсальное развёртывание: на системах с NVIDIA GPU используется CUDA-ускорение через NVIDIA Container Toolkit, на системах без GPU автоматически активируется CPU-режим. Конфигурация сервиса осуществляется исключительно через переменные окружения без необходимости пересборки образа.

Запуск ASGI-приложения FastAPI обеспечивается сервером Uvicorn,

основанным на uvloop (быстрая реализация asyncio event loop на базе libuv) и httptools (HTTP-парсер на Cython). Uvicorn обрабатывает одновременные соединения в рамках единого процесса Python через кооперативную многозадачность asyncio.

3.2 Язык и библиотеки

Серверная логика реализована на языке Python версии 3.13 с применением статической типизации через type annotations. Код структурирован в классы и функции с явным указанием типов аргументов и возвращаемых значений, что повышает читаемость и позволяет выявлять ошибки на этапе статического анализа через инструменты mypy или pyright.

Асинхронное программирование базируется на стандартном модуле asyncio, входящем в состав стандартной библиотеки Python. Используются примитивы синхронизации asyncio.Semaphore для ограничения количества одновременных операций, asyncio.Task для запуска фоновых задач, asyncio.shield для защиты критических участков кода от преждевременной отмены при разрыве клиентского соединения. Все HTTP-обработчики объявлены как async def функции, что позволяет освобождать event loop во время I/O операций.

Многопоточная синхронизация реализована через модуль threading стандартной библиотеки. Применены threading.Lock для защиты критических секций при изменении разделяемого состояния, threading.Semaphore для контроля количества одновременно выполняющихся блокирующих операций инференса, threading.Condition для координации между потоками при переключении моделей с GPU на CPU. Condition variable обеспечивает атомарность операции wait-and-block с автоматическим освобождением и повторным захватом мьютекса.

Передача контекстной информации между асинхронными вызовами выполнена через contextvars – механизм context-local storage, добавленный в Python 3.7. ContextVar для идентификатора запроса автоматически изолирован между параллельными Task, что позволяет корректно отслеживать принадлежность логов к конкретному HTTP-запросу без явной передачи request_id через аргументы функций.

Система логирования построена на модуле logging с настройкой Formatter для структурированного вывода, Filter для внедрения контекстных переменных в LogRecord, Handler для направления вывода в множественные destinations. RotatingFileHandler обеспечивает автоматическую ротацию файлов при достижении порогового размера с сохранением заданного числа архивных копий.

Модуль faulthandler, активируемый при старте приложения, перехватывает критические ошибки на уровне C-расширений (segmentation fault, stack overflow) и выводит полную трассировку всех потоков Python в указанный файл. Это критично для диагностики падений, вызванных нативными библиотеками PyTorch или CUDA-драйверами.

4 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ПРОГРАММНОГО МОДУЛЯ

4.1 Функциональность сервиса

Разработанный HTTP-сервис GigaServer реализует автоматизированную обработку аудиозаписей диалогов с выделением реплик отдельных спикеров и транскрипцией речевого содержимого. Входными данными служит аудиофайл произвольного формата, передаваемый через HTTP POST запрос в multipart/form-data кодировке. Сервис выполняет последовательность операций: декодирование и нормализация аудио (ресемплинг до 16 кГц, конвертация в моно PCM), диаризация с использованием модели Sortformer для определения временных границ реплик и идентификации спикеров, распознавание речи моделью GigaAM для каждого сегмента, формирование результата в виде временных меток, идентификаторов спикеров и текстового содержимого.

API сервиса совместим со спецификацией OpenAI для эндпоинта распознавания речи. Поддерживаются два URL-пути: /audio/transcriptions и /v1/audio/transcriptions, оба обрабатываются идентичной логикой. Клиент может запросить результат в одном из форматов: json возвращает объект с полем text, содержащим многострочный текст с разметкой спикеров; text возвращает тот же текст в plain/text формате; debug предоставляет расширенную диагностическую информацию включая промежуточные результаты диаризации, метрики производительности и информацию о fallback-событиях.

Административный эндпоинт GET /health предоставляет информацию о текущем состоянии сервиса в JSON формате: статус готовности (ok если модели загружены, loading в процессе инициализации), используемые модели и устройство (CPU или CUDA), конфигурация пула обработки (размер, таймаут очереди), текущая загрузка (количество занятых и свободных слотов), накопленная статистика запросов с момента старта.

Ключевой особенностью реализации является автоматическая адаптация к доступным вычислительным ресурсам. При наличии CUDA-совместимого GPU сервис использует GPU-ускорение для обеих моделей. При возникновении CUDA-ошибок (недостаток видеопамяти, драйверные проблемы, аппаратные сбои) происходит автоматическое переключение на CPU-реализацию, обеспечивая непрерывность обслуживания при деградации производительности.

4.2 Декомпозиция на модули

Центральным результатом работы стала декомпозиция монолитного кода на слабосвязанные модули. Архитектура организована в соответствии с layered architecture pattern: слой конфигурации и инфраструктуры, транспортный слой, слой бизнес-логики, слой вспомогательных утилит.

Модуль `config.py` инкапсулирует всю конфигурационную логику. Определены константы для параметров модели (`SORTFORMER_MODEL`, `MODEL_NAME`, `SORTFORMER_DEVICE`, `SORTFORMER_ASR_BATCH_SIZE`), параметров пула инференса (`INFER_POOL_SIZE`, `INFER_TIMEOUT_S`, `QUEUE_TIMEOUT_S`), настроек логирования (`LOG_LEVEL`, `LOG_DIR`, `LOG_FILE`, `LOG_MAX_BYTES`, `LOG_BACKUP_COUNT`). Каждый параметр читается из переменных окружения с определённым значением по умолчанию. Модуль содержит функцию `configure_logging()`, которая инициализирует logging subsystem: создаёт `StreamHandler` для stdout и `RotatingFileHandler` для файла, настраивает единый `Formatter` со структурированным форматом записей, регистрирует `RequestIdFilter` для автоматического внедрения идентификатора запроса. Дополнительно активируется `faulthandler` с направлением вывода в отдельный файл и устанавливаются обработчики `sys.excepthook` и `threading.excepthook` для логирования необработанных исключений.

Модуль `server.py` реализует HTTP API и координацию запросов. Класс `FastAPI` инициализируется с параметром `lifespan`, где определён `async context manager` для выполнения startup и shutdown логики. При startup создаётся экземпляр `Metrics`, инициализируется `asyncio.Semaphore` с размером `INFER_POOL_SIZE`, вызывается `init_semaphore()` для инициализации синхронного семафора, запускается асинхронная задача загрузки моделей через `asyncio.to_thread()`. HTTP middleware `log_requests` оборачивает каждый запрос: извлекает `X-Request-Id` заголовок или генерирует новый идентификатор, устанавливает его в `contextvars` через `set_request_id()`, вызывает обработчик запроса, сбрасывает контекст через `reset_request_id()`, добавляет `X-Request-Id` в заголовки ответа, логирует метрики времени выполнения. Функция `_handle()` инкапсулирует полный цикл обработки одного запроса: учёт в метриках, ожидание слота в очереди с обработкой таймаутов, чтение тела запроса, создание защищённой `asyncio.Task` для вызова `transcribe_sync()`, обработка таймаута инференса через `asyncio.wait_for()` с использованием `asyncio.shield()` для предотвращения отмены фоновой задачи, форматирование и возврат результата.

Модуль `pipeline.py` управляет жизненным циклом ML-моделей и координацией инференса. Глобальные переменные `_sortformer_model`, `_asr_model` и `_runtime_device` хранят загруженные модели и текущий режим работы. Функция `load_models()` выполняет загрузку с обработкой fallback: вызывает `select_runtime_device()` для определения целевого устройства, вызывает `_load_models_on()` для загрузки обеих моделей, при CUDA-ошибке в auto-режиме логирует исключение, вызывает `_release_cuda_models()` для очистки памяти, повторяет загрузку на CPU. Функция `transcribe_sync()` координирует полный pipeline: вызывает `write_wav_temp()` для конвертации аудио, считывает длительность через `wav_duration()`, вызывает `run_with_timeout()` для выполнения инференса с watchdog, обрабатывает CUDA-ошибки через `_fallback_to_cpu()` при необходимости. Функция `_fallback_to_cpu()` реализует координированное переключение: устанавливает

`_fallback_in_progress` флаг, захватывает `_inference_condition`, ожидает обнуления `_active_inferences`, выгружает CUDA-модели, загружает CPU-версии, сбрасывает флаг и оповещает ожидающие потоки. Функции `_begin_inference()` и `_end_inference()` обеспечивают корректный подсчёт активных операций: `_begin_inference()` блокируется пока `_fallback_in_progress=True`, инкрементирует счётчик; `_end_inference()` декрементирует счётчик и оповещает condition variable.

Модуль `audio.py` содержит low-level операции с аудиофайлами. Функция `read_wav_header()` выполняет парсинг RIFF/WAVE заголовка: проверяет сигнатуру, ищет `fmt chunk`, извлекает `audio_format`, `channels`, `sample_rate`, `bits_per_sample`. Функция `is_16k_pcm()` проверяет соответствие параметров требуемому формату. Функция `write_wav_temp()` реализует оптимизированный путь: если входной файл уже в целевом формате, он копируется без конвертации; в противном случае создаётся временный входной файл, запускается `ffmpeg` с параметрами конвертации, проверяется `return code`, удаляется входной временный файл, возвращается путь к выходному файлу. Функция `wav_duration()` использует `wave.open()` для быстрого чтения метаданных без декодирования сэмплов.

Модуль `formatting.py` трансформирует результаты распознавания в форматы вывода. Функция `format_result()` принимает строку транскрипции, массив сегментов, метрики и параметр `response_format`. Для `json` и `verbose_json` создаётся `JSONResponse` с объектом `{"text": "..."}` где значение содержит построчный текст с `\n`-разделителями. Для `text` создаётся `PlainTextResponse` с тем же текстом. Для `debug` создаётся полный JSON с дополнительными полями: `segments` (детализированные временные интервалы), `speakers` (список уникальных идентификаторов), `diarization` (промежуточные данные диаризации, включая `raw_turns`, `clean_turns`, `removed_inversions`), `performance` (метрики времени выполнения, `realtime factor`, размеры). Функция `_performance_headers()` преобразует словарь метрик в набор HTTP-заголовков `X-ASR-*` для потокового доступа к метрикам без парсинга тела ответа.

Модуль `metrics.py` предоставляет потокобезопасный счётчик событий. Класс `Metrics` инкапсулирует все переменные состояния: `requests_total`, `requests_ok`, `requests_failed`, `requests_failed_by_stage`, `active`, `waiting`, `last_elapsed_s`, `total_audio_duration_s`. Все методы изменения состояния (`request_start`, `request_acquired`, `request_finished`, `request_done`, `request_rejected`, `request_errored`) защищены `threading.Lock` для обеспечения атомарности. Метод `snapshot()` возвращает копию текущего состояния для безопасного чтения из внешнего кода. Расчётные метрики (`avg_realtime_factor`, `queue_depth`, `uptime_s`) вычисляются в момент вызова `snapshot()`.

Файл `main.py` сокращён до минималистичной точки входа: импортирует `app` из `server`, при запуске как `__main__` вызывает `uvicorn.run()` с параметрами хоста, порта и отключением встроенного логирования `uvicorn` в пользу кастомной конфигурации.

4.3 Контроль нагрузки

Разработана двухуровневая система управления нагрузкой, предотвращающая исчерпание ресурсов при интенсивном трафике. Верхний уровень реализован на базе `asyncio.Semaphore` в модуле `server.py`. При получении HTTP-запроса функция `_handle()` вызывает `acquire()` на семафоре до начала любой обработки. Семафор инициализируется при `startup` с размером `INFER_POOL_SIZE`, ограничивая количество одновременно обслуживаемых запросов. Поддерживаются три режима поведения при отсутствии свободных слотов: `QUEUE_TIMEOUT_S < 0` активирует блокирующий `acquire()` без таймаута, запрос ожидает неограниченно долго; `QUEUE_TIMEOUT_S = 0` активирует неблокирующий `acquire(blocking=False)`, при занятости всех слотов немедленно возвращается HTTP 429 Too Many Requests с заголовком `Retry-After`; `QUEUE_TIMEOUT_S > 0` оборачивает `acquire()` в `asyncio.wait_for()`, при превышении таймаута генерируется `asyncio.TimeoutError` который перехватывается и преобразуется в HTTP 503 Service Unavailable с `Retry-After`.

Нижний уровень реализован через `threading.Semaphore` в модуле `pipeline.py`. Функция `run_with_timeout()` проверяет параметр `slot_acquired`: если `False`, выполняет `acquire()` на синхронном семафоре перед запуском инференса. Это обеспечивает корректную работу при прямых синхронных вызовах `transcribe_sync()` без HTTP-слоя. При вызовах из HTTP-обработчика `slot_acquired=True` предотвращает двойной учёт запроса в очереди. Семафор освобождается в блоке `finally` после завершения инференса независимо от результата (успех, ошибка, таймаут).

Метрики `Metrics` обеспечивают наблюдаемость состояния очереди. Метод `request_start()` инкрементирует `waiting` при постановке запроса в очередь. Метод `request_acquired()` переводит счётчик из `waiting` в `active` при получении слота. Метод `request_finished()` декрементирует `active` при завершении обработки. Метод `request_rejected()` декрементирует `waiting` при отказе в обслуживании по таймауту. Это позволяет эндпоинту `/health` показывать `requests_active` (текущее число обрабатываемых запросов), `requests_waiting` (текущее число ожидающих), `queue_depth` (суммарная нагрузка).

Таймаут инференса реализован через отдельный `watchdog`-поток в `run_with_timeout()`. Создаётся `threading.Thread` с целевой функцией `worker()`, которая вызывает `callback` инференса и сохраняет результат или исключение в общих переменных. Главный поток вызывает `thread.join(INFER_TIMEOUT_S)`. Если `join()` возвращается до таймаута, результат извлекается из общих переменных. Если таймаут превышен, генерируется `TimeoutError` для клиента, но фоновый поток продолжает выполнение до завершения инференса, удерживая слот пула. Это предотвращает параллельное выполнение сверх `INFER_POOL_SIZE` при таймаутах. В HTTP-слое применяется `asyncio.shield()` для защиты задачи `asyncio.to_thread()`: даже если клиент разрывает соединение, фоновая задача

должна завершиться и освободить слот перед приёмом нового запроса.

4.4 Инфраструктура наблюдаемости

Реализована инфраструктура наблюдаемости, обеспечивающая детальную диагностику поведения сервиса в production. Структурированное логирование построено на стандартном logging framework с кастомизированным Formatter. Формат записи определён как: "{timestamp}.{milliseconds} [{level}] [pid={process} thread={thread_name} request={request_id}] {logger_name}: {message}". Фильтр RequestIdFilter автоматически внедряет значение request_id из contextvars в каждую LogRecord перед форматированием, что обеспечивает автоматическую корреляцию всех записей лога в рамках одного запроса без явного прокидывания идентификатора через аргументы.

RotatingFileHandler настроен на ротацию по размеру: при достижении LOG_MAX_BYTES (по умолчанию 50 МБ) текущий файл переименовывается в asr.log.1, создаётся новый asr.log. При следующей ротации asr.log.1 → asr.log.2, asr.log → asr.log.1. Файлы старше LOG_BACKUP_COUNT автоматически удаляются. Дополнительный StreamHandler дублирует вывод в stdout для интеграции с системами агрегации логов (ELK stack, Loki).

Критические падения процесса перехватываются через faulthandler. При старте приложения вызывается faulthandler.enable(file=fault_log_handle) для активации обработчика segfaults и других низкоуровневых ошибок. При возникновении критической ошибки faulthandler записывает полную трассировку всех потоков Python в указанный файл перед завершением процесса. Это критично для диагностики проблем в нативных расширениях PyTorch и CUDA-библиотеках, где стандартный механизм исключений Python не применим.

Идентификаторы запросов генерируются в middleware log_requests: сначала проверяется наличие заголовка X-Request-Id в запросе (для поддержки сквозной трассировки через границы сервисов), при отсутствии генерируется uuid.uuid4().hex[:12]. Идентификатор сохраняется в contextvars через set_request_id(), что делает его доступным во всех синхронных и асинхронных вызовах в рамках обработки данного запроса. При завершении обработки вызывается reset_request_id() для очистки контекста. Идентификатор добавляется в заголовки ответа для связывания клиентских и серверных логов.

Детальное логирование покрывает все этапы обработки: получение запроса (метод, путь, размер тела, request_id), начало конвертации аудио (входной размер, формат), результат конвертации (выходной размер, соотношение размеров, время ffmpeg, предупреждения stderr), загрузка моделей (устройство, время инициализации), начало инференса (длительность аудио, количество каналов), результат диаризации (количество спикеров, количество сегментов), результат ASR (количество символов, realtime factor), события fallback (причина ошибки CUDA, время переключения), завершение запроса (HTTP код, общее время, throughput).

Эндпоинт `/health` агрегирует текущие метрики через метод `Metrics.snapshot()`. Возвращаемый JSON содержит: `status` (ok/loading), `model` (название ASR модели), `diarization` (backend, модель диаризации, устройство, batch size), `pool_size`, `slots_free` (вычисляется как `pool_size - requests_active`), `queue_timeout_s`, `requests` (total, ok, failed, failed_by_stage с разбивкой по decode/inference/queue/handler). Метрики производительности включены в HTTP-заголовки ответов: X-ASR-Request-Elapsed-S (полное время обработки), X-ASR-Pipeline-Elapsed-S (время pure инференса), X-ASR-Audio-Duration-S (длительность аудио), X-ASR-Realtime-Factor (соотношение длительности к времени обработки), X-ASR-Segments (количество распознанных сегментов), X-ASR-Chars (количество символов в транскрипции), X-ASR-Stage-Convert-S и X-ASR-Stage-Diarization-And-ASR-S (детализация по стадиям).

4.5 Доработка микросервисов платформы Century

Параллельно с развитием HTTP-сервера GigaServer выполнялась работа по анализу и улучшению ключевых микросервисов корпоративной AI-платформы Century. Данная работа охватывала изучение архитектуры системы, выявление узких мест и реализацию необходимых доработок.

Анализ микросервиса управления агентами (AgentBuilder). Проведено изучение структуры хранения конфигураций агентов, их промптов, параметров взаимодействия с языковыми моделями и доступных инструментов. Выявлены возможности для оптимизации запросов к базе данных при получении полной конфигурации агента со всеми связанными сущностями. Реализовано использование SQL JOIN операций вместо множественных последовательных запросов, что сократило количество обращений к базе данных и уменьшило латентность API. Улучшена валидация входных параметров при создании и обновлении агентов для предотвращения сохранения некорректных конфигураций. Расширена функциональность API для поддержки дополнительных параметров фильтрации и сортировки при получении списков агентов.

Оптимизация микросервиса истории сообщений (ConversationService). Изучена схема хранения диалогов пользователей с агентами, механизмы управления контекстом беседы и обработки вложений. Выявлена проблема N+1 запросов при получении истории диалога с метаданными сообщений. Реализована загрузка связанных данных через eager loading с использованием подзапросов и JOIN операций. Оптимизированы индексы таблицы сообщений для ускорения выборки по часто используемым условиям фильтрации (временной диапазон, идентификаторы диалогов, пользователи). Улучшена обработка граничных случаев при работе с вложениями большого размера: добавлены проверки размера файлов, корректная обработка таймаутов при загрузке из объектного хранилища, информативные сообщения об ошибках для клиентов. Реализована пагинация результатов при получении длинных историй диалогов для снижения нагрузки на сеть и клиентские приложения.

Развитие микросервиса мультиагентов. Проанализированы механизмы

координации взаимодействия нескольких агентов в рамках решения одной задачи, передачи контекста между агентами и агрегации результатов. Реализованы улучшения для поддержки новых сценариев работы платформы: расширен набор стратегий выбора следующего агента в цепочке (sequential, parallel, conditional), улучшена трассировка выполнения multi-agent сценариев через структурированное логирование с идентификаторами сессий, добавлена обработка частичных сбоев (fallback на альтернативных агентов при недоступности основного). Оптимизирована сериализация и десериализация промежуточных результатов работы агентов для снижения overhead при передаче больших объёмов данных между компонентами.

Интеграция с сервисом транскрипции. В рамках работы над HTTP-сервером GigaServer выполнена его интеграция в экосистему платформы Century через микросервис transcription_service. Реализован клиентский адаптер для вызова API распознавания речи из других компонентов платформы, обработка различных форматов ответов (json, text, debug), корректная передача метаданных запросов (идентификаторы пользователей, tenant isolation). Добавлена обработка ошибок и retry-логика для повышения устойчивости к временным сбоям сервиса распознавания. Реализована интеграция результатов транскрипции в контекст диалогов для использования в последующих запросах к языковым моделям.

Общие улучшения кодовой базы. Выполнен рефакторинг дублирующейся логики работы с базой данных: выделены переиспользуемые функции для типовых операций (получение записи по ID с обработкой отсутствия, пагинированные выборки, bulk операции). Улучшена структура модулей: явное разделение слоёв транспорта (HTTP handlers), бизнес-логики (services) и доступа к данным (repositories). Вынесены конфигурационные параметры в переменные окружения для упрощения развёртывания в различных окружениях. Добавлены типовые аннотации для улучшения статического анализа кода и читаемости. Расширено покрытие unit-тестами критических компонентов: валидация бизнес-логики, граничные случаи обработки данных, корректность SQL-запросов.

Результатом работы стало повышение производительности ключевых API-эндпоинтов платформы (снижение латентности на 30-50% для операций получения данных за счёт оптимизации запросов), улучшение стабильности системы через более полную обработку ошибок и граничных случаев, расширение функциональных возможностей для поддержки новых сценариев использования, повышение поддерживаемости кодовой базы через рефакторинг и улучшение структуры кода.

5 ТЕСТИРОВАНИЕ РАЗРАБОТАННОГО РЕШЕНИЯ

5.1 Верификация архитектуры

Тестирование модульной структуры проводилось на локальной машине разработчика с воспроизведением конфигурации production-окружения. Первоначальная верификация включала статический анализ: запуск приложения с проверкой отсутствия `import`-ошибок, циклических зависимостей между модулями, корректной инициализации всех компонентов при `startup`. Анализ показал полную независимость модулей: изменение реализации `audio.py` не требует модификаций в `server.py` или `pipeline.py`, добавление нового формата вывода в `formatting.py` изолировано в рамках одного модуля.

Функциональность логирования тестировалась через одновременную отправку нескольких запросов с использованием `curl` в фоновых `bash`-процессах. Каждый запрос получал уникальный `request_id`, который появлялся во всех связанных записях лога от момента получения HTTP-запроса до возврата ответа. `Grep`-поиск по конкретному `request_id` в файле логов извлекал только записи данного запроса, подтверждая корректность трассировки. Проверено смешение записей разных запросов во времени (`timestamp` записей перемежаются), но изоляция по `request_id` сохраняется.

Механизм ротации логов верифицирован через генерацию большого объёма логов: запущен цикл отправки запросов до превышения `LOG_MAX_BYTES`. При достижении порога существующий `asr.log` переименован в `asr.log.1`, создан новый пустой `asr.log`. Записи продолжились в новый файл без потери данных. Последующие ротации корректно сдвигали нумерацию файлов (`.1 → .2`, `.2 → .3`), файлы с номером выше `LOG_BACKUP_COUNT` автоматически удалялись.

Обработчик критических падений `faulthandler` протестирован через искусственное создание `segmentation fault` с использованием `ctypes` для вызова невалидного адреса памяти. При падении процесса `faulthandler` корректно записал полную трассировку всех потоков в файл `faulthandler.log` перед завершением. Трассировка включала не только основной поток, но и все фоновые потоки обработки запросов и `watchdog`-потоки инференса.

5.2 Валидация системы очередей

Система управления нагрузкой тестировалась путём отправки большого количества запросов, чем размер пула. Использован скрипт `bash` для параллельного запуска `N` экземпляров `curl` в фоновом режиме с ожиданием завершения всех процессов через `wait`. При `INFER_POOL_SIZE=2` и 10 одновременных запросах наблюдалось следующее поведение: первые 2 запроса немедленно переходили в обработку (`requests_active=2`), остальные 8 переходили в состояние ожидания (`requests_waiting=8`), по мере завершения активных запросов счётчик `requests_waiting` уменьшался, а освободившиеся

слоты занимались следующими запросами из очереди. Финальное состояние: все 10 запросов успешно обработаны, `requests_active=0`, `requests_waiting=0`.

Режим немедленного отказа проверен при `QUEUE_TIMEOUT_S=0`. При отправке 5 запросов с `pool_size=2` первые два получили HTTP 200 OK после обработки, остальные три мгновенно получили HTTP 429 Too Many Requests с заголовком `Retry-After: 1`. Повторная отправка отклонённых запросов через 2 секунды (после освобождения слотов) завершилась успешно.

Режим ожидания с таймаутом тестировался при `QUEUE_TIMEOUT_S=30` с длинным входным файлом (>2 минут аудио). При отправке 4 запросов с `pool_size=2` первые два начали обработку, третий и четвёртый перешли в ожидание. Через 30 секунд ожидающие запросы получили HTTP 503 Service Unavailable с `Retry-After: 30`, активные запросы продолжили обработку до завершения.

Корректность координации между `asyncio.Semaphore` и `threading.Semaphore` проверена через логирование счётчиков на разных уровнях. При `slot_acquired=True` (вызов из HTTP-слоя) синхронный семафор не блокировал выполнение, значение `requests_active` корректно отражало количество одновременных инференсов и никогда не превышало `INFER_POOL_SIZE` даже при нагрузке 20 одновременных HTTP-запросов.

Watchdog-таймер инференса тестировался при `INFER_TIMEOUT_S=10` с аудиофайлом длительностью 90 секунд. Клиент получил HTTP 504 Gateway Timeout ровно через 10 секунд после отправки запроса. В логах сервера появилась запись "Inference timed out; retaining pool slot until worker exits". Слот пула оставался занятым (`requests_active` не уменьшалось), следующий запрос ожидал освобождения слота. Через ~40 секунд после таймаута (общее время инференса длинного файла на CPU) фоновый поток завершился, `requests_active` декрементировалось, очередной запрос начал обработку. Это подтвердило, что таймаут защищает клиента от долгого ожидания, но не создаёт параллельное выполнение сверх лимита.

Обработка разрыва соединения клиентом проверена через отправку запроса с немедленной отменой (Ctrl+C на клиенте). В логах сервера появилась запись о разрыве соединения, но обработка запроса продолжилась до завершения инференса благодаря `asyncio.shield()`. Слот корректно освободился после завершения, метрика `requests_failed` не инкрементировалась (клиентская отмена не считается серверной ошибкой).

Нагрузочное тестирование выполнено через ApacheBench с параметрами: `ab -n 100 -c 20 -p audio.wav -T "multipart/form-data; boundary=..." http://localhost:9876/audio/transcriptions`. При `INFER_POOL_SIZE=3` все 100 запросов успешно обработаны без ошибок 5xx. Среднее время ответа составило ~25 секунд (время инференса одного запроса ~7 секунд × queue depth ~3). Пиковое значение `requests_active` во время теста равнялось 3, подтверждая корректность ограничения пула.

5.3 Проверка отказоустойчивости

Механизм автоматического fallback с GPU на CPU тестировался в двух сценариях: при инициализации и во время runtime. Для симуляции CUDA-ошибки при инициализации использована подмена переменной окружения `CUDA_VISIBLE_DEVICES="999"` (несуществующий GPU). При запуске с `SORTFORMER_DEVICE=auto` сервис попытался загрузить модели на CUDA, получил ошибку "CUDA device 999 is not available", автоматически перешёл на CPU-режим. В логах зафиксирована последовательность: "CUDA initialization failed; retrying on CPU", "Models loaded successfully: device=cpu". Эндпоинт `/health` показал `diarization.device=cpu, status=ok`. Последующие запросы обрабатывались корректно на CPU.

Runtime fallback тестировался через искусственное создание CUDA out of memory ситуации: отправлено множество одновременных запросов с длинными аудиофайлами для исчерпания видеопамати. При возникновении CUDA OOM во время инференса одного из запросов сработал механизм `_fallback_to_cpu()`: установлен флаг `_fallback_in_progress`, заблокированы новые запросы, ожидание завершения активных инференсов (~5 секунд для завершения других запросов на GPU), выгрузка GPU-моделей с `torch.cuda.empty_cache()`, загрузка CPU-моделей, сброс флага и возобновление обработки. Запрос, вызвавший ошибку, автоматически повторён на CPU-моделях и завершился успешно. Все последующие запросы обрабатывались на CPU без дополнительных ошибок. Время недоступности сервиса составило ~8 секунд (время переключения моделей).

Координация через `threading.Condition` проверена через детальное логирование: при `_begin_inference()` во время `fallback_in_progress=True` поток корректно блокировался на `condition.wait()`, после завершения fallback получал оповещение `condition.notify_all()` и продолжал выполнение. Счётчик `_active_inferences` корректно достигал нуля перед началом замены моделей, исключая race condition между выгрузкой старых и загрузкой новых моделей.

Тестирование подтвердило, что разработанная система управления очередями и отказоустойчивости обеспечивает стабильную работу сервиса под нагрузкой, корректно обрабатывает граничные случаи (таймауты, разрывы соединений, ошибки CUDA) и предоставляет детальную диагностическую информацию для мониторинга состояния в production-окружении.

5.4 Тестирование доработок микросервисов платформы

Тестирование изменений в микросервисах платформы Century проводилось на локальном окружении разработчика с использованием `docker-compose` для развёртывания полного стека зависимостей (PostgreSQL, RabbitMQ, MinIO, связанные сервисы). Применялся комплексный подход: unit-тесты для изолированной проверки бизнес-логики, интеграционные тесты для валидации взаимодействия с базой данных и внешними сервисами, ручное

функциональное тестирование через API.

Тестирование оптимизации запросов. Для проверки эффективности оптимизации запросов к базе данных использовались SQL query logs с включённым профилированием времени выполнения. До оптимизации операция получения полной конфигурации агента со всеми связанными инструментами и параметрами генерировала 15-20 отдельных SELECT запросов (N+1 проблема). После рефакторинга с использованием JOIN и eager loading количество запросов сократилось до 2-3, а общее время выполнения операции уменьшилось с ~150ms до ~45ms (снижение на 70%). Аналогичные улучшения получены для операций получения истории диалогов: время загрузки 100 сообщений с метаданными сократилось с ~300ms до ~80ms.

Проверка корректности добавленных индексов. Использован анализ планов выполнения запросов через EXPLAIN ANALYZE в PostgreSQL. До добавления индексов запросы с фильтрацией по временному диапазону использовали Sequential Scan с временем выполнения ~200ms на таблице с 100k записей. После создания составного индекса по полям (conversation_id, created_at) запросы стали использовать Index Scan с временем выполнения ~5ms. Валидировано отсутствие регрессии производительности операций записи: измерено время вставки 1000 новых сообщений до и после добавления индексов, overhead составил менее 5% (приемлемо для оптимизации чтения).

Тестирование улучшенной обработки ошибок. Проверена корректность обработки граничных случаев через искусственное создание условий ошибок: отключение внешних сервисов (MinIO для тестирования timeout при загрузке файлов, RabbitMQ для проверки retry-логики), отправка невалидных входных данных (некорректные форматы, отсутствующие обязательные поля, значения вне допустимого диапазона), проверка поведения при отсутствии запрашиваемых сущностей в базе данных. Во всех случаях API возвращали корректные HTTP статус-коды (400 для клиентских ошибок, 404 для отсутствующих ресурсов, 503 для недоступности внешних зависимостей) и информативные сообщения об ошибках в теле ответа. Подтверждено отсутствие утечек внутренних деталей реализации (стектрейсов, SQL-запросов) в production режиме.

Валидация расширенной функциональности API. Проверена корректность работы новых параметров фильтрации и пагинации через серию тестовых запросов: получение списка агентов с фильтрацией по типу, дате создания, владельцу; сортировка по различным полям (имя, дата изменения) в прямом и обратном порядке; пагинация с различными размерами страниц (10, 50, 100 элементов), проверка корректности метаданных (total_count, has_next_page). Протестирована устойчивость к граничным значениям параметров (page_size=0, page_size=1000000, отрицательные номера страниц). API корректно обрабатывали некорректные значения с возвратом HTTP 400 и описанием проблемы.

Интеграционное тестирование multi-agent сценариев. Разработаны тестовые сценарии для проверки координации нескольких агентов: последовательное выполнение цепочки из 3 агентов с передачей контекста,

параллельное выполнение 2 независимых агентов с агрегацией результатов, условное ветвление (выбор следующего агента на основе результата предыдущего). Валидирована трассируемость выполнения: каждый шаг записывается в лог с единым `session_id`, позволяя восстановить полный путь выполнения. Проверена обработка частичных сбоев: при недоступности одного агента в цепочке система корректно переключается на fallback-агента без прерывания всего сценария.

Unit-тестирование бизнес-логики. Разработан набор unit-тестов для изолированной проверки рефакторизованных компонентов: валидация входных данных при создании/обновлении сущностей, корректность преобразования моделей данных между слоями (`DTO` → `entity` → `DTO`), логика выбора стратегий в multi-agent сценариях. Использованы моки для изоляции от внешних зависимостей (база данных, HTTP-клиенты). Достигнуто покрытие тестами критических компонентов на уровне >80% по метрике `line coverage`.

Проверка обратной совместимости. Запущен полный регрессионный набор существующих тестов платформы для подтверждения отсутствия `breaking changes`. Все 450+ существующих интеграционных тестов прошли успешно, подтверждая сохранение контрактов API и поведения системы. Выполнено ручное тестирование критических пользовательских сценариев через веб-интерфейс платформы: создание агента, отправка сообщений в диалог, просмотр истории, выполнение multi-agent задач. Функциональность работала корректно, производительность улучшилась (заметное ускорение загрузки истории диалогов и списков агентов).

Комплексное тестирование подтвердило корректность всех внесённых изменений в микросервисы платформы Century, отсутствие регрессий в существующей функциональности и достижение поставленных целей по улучшению производительности, стабильности и поддерживаемости кода.

ЗАКЛЮЧЕНИЕ

В результате прохождения производственной практики в ООО «СТЭКЛЭВЭЛ ГРУПП» были достигнуты все поставленные цели и выполнены все задачи.

Изучена архитектура корпоративной AI-платформы Century. Проанализированы принципы взаимодействия компонентов через REST API, паттерны организации кода, методы работы с базами данных в распределённых системах. Выполнен анализ и доработка ключевых микросервисов платформы: сервиса управления агентами, сервиса истории сообщений, сервиса мультиагентов. Реализованы оптимизации производительности запросов к базе данных, улучшена обработка ошибок, расширена функциональность API.

Проведён масштабный рефакторинг HTTP-сервера GigaServer для обработки аудио. Монолитный файл разделён на специализированные модули с чёткой ответственностью: конфигурация, транспорт, управление моделями, работа с аудио, форматирование ответов, сбор метрик. Реализована модульная архитектура. Разработана система управления очередями запросов с двухуровневой архитектурой: асинхронная очередь на уровне HTTP и синхронная очередь на уровне инференса. Реализована поддержка различных режимов ожидания и корректная координация между уровнями.

Система обеспечивает контроль нагрузки на сервис и предотвращает исчерпание ресурсов при интенсивном трафике. Внедрена комплексная система логирования с записью структурированных логов в stdout и ротационный файл. Реализована сквозная трассировка запросов через идентификаторы, логирование критических падений через fault handler, автоматическое добавление метрик производительности в заголовки HTTP-ответов. Разработан механизм автоматического переключения с GPU на CPU при возникновении CUDA-специфичных ошибок. Механизм обеспечивает отказоустойчивость сервиса: при нестабильности GPU-среды система продолжает работать на CPU без простоя.

Проведено комплексное тестирование всех разработанных компонентов. Подтверждена корректность модульной архитектуры, работы системы управления очередями под нагрузкой, логирования и трассировки запросов, механизма автоматического fallback.

Производственная практика позволила получить ценный опыт работы с современными технологиями разработки серверных приложений: Python, FastAPI, asyncio, PyTorch, Docker. Освоены best practices проектирования микросервисных архитектур, принципы separation of concerns, методы обеспечения отказоустойчивости и наблюдаемости распределённых систем, техники оптимизации использования вычислительных ресурсов для ML-моделей.

Полученные в ходе практики знания и навыки являются актуальными для современной индустрии разработки программного обеспечения и будут применены в дальнейшей профессиональной деятельности.

СПИСОК ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ

- [1] Официальный сайт ООО «СТЭКЛЭВЭЛ ГРУПП» [Электронный ресурс]. – Режим доступа: <https://stacklevel.group/> – Дата доступа: 30.06.2026.
- [2] FastAPI Documentation. Modern web framework for building APIs with Python [Электронный ресурс]. – Режим доступа: <https://fastapi.tiangolo.com/> – Дата доступа: 30.06.2026.
- [3] NVIDIA NeMo Toolkit. Conversational AI toolkit [Электронный ресурс]. – Режим доступа: <https://github.com/NVIDIA/NeMo> – Дата доступа: 01.07.2026.
- [4] ONNX Runtime Documentation. Cross-platform inference engine [Электронный ресурс]. – Режим доступа: <https://onnxruntime.ai/docs/> – Дата доступа: 01.07.2026.
- [5] FFmpeg Documentation. Multimedia framework [Электронный ресурс]. – Режим доступа: <https://ffmpeg.org/documentation.html> – Дата доступа: 01.07.2026.
- [6] PostgreSQL Documentation. Advanced open source database [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/> – Дата доступа: 01.07.2026.
- [7] Docker Documentation. Containerization platform [Электронный ресурс]. – Режим доступа: <https://docs.docker.com/> – Дата доступа: 01.07.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программного кода

```
"""OpenAI-compatible HTTP transport for the Sortformer/GigaAM
pipeline."""

from __future__ import annotations

import asyncio
import logging
import os
import platform
import sys
import time
import uuid
from contextlib import asynccontextmanager

from fastapi import FastAPI, File, Form, HTTPException, Request,
UploadFile
from fastapi.responses import Response

from config import INFER_POOL_SIZE, INFER_TIMEOUT_S, LOG_TRANSCRIPTS,
MODEL_NAME, QUEUE_TIMEOUT_S, queue_retry_after, reset_request_id,
set_request_id, short_text
from formatting import format_result
from metrics import Metrics
from pipeline import init_semaphore, is_model_loaded, load_models,
runtime_info, transcribe_sync

logger = logging.getLogger("asr")
_metrics: Metrics | None = None
_pipeline_semaphore: asyncio.Semaphore | None = None

@asynccontextmanager
async def lifespan(_: FastAPI):
    global _metrics, _pipeline_semaphore
    _metrics = Metrics(started_at=time.time())
    _pipeline_semaphore = asyncio.Semaphore(max(INFER_POOL_SIZE, 1))
    init_semaphore()
    logger.info("Startup environment: python=%s platform=%s pid=%d",
sys.version.replace("\n", " "), platform.platform(), os.getpid())
    device = await asyncio.to_thread(load_models)
    logger.info("Models loaded successfully: GigaAM=%s Sortformer=%s",
MODEL_NAME, device)
    yield

app = FastAPI(title="GigaServer Sortformer ASR", lifespan=lifespan)

@app.middleware("http")
async def log_requests(request: Request, call_next) -> Response:
    request_id = request.headers.get("x-request-id") or
uuid.uuid4().hex[:12]
    token = set_request_id(request_id)
    started = time.perf_counter()
    try:
        response = await call_next(request)
    except Exception:
        logger.exception("HTTP request crashed: method=%s path=%s",
```

```

request.method, request.url.path)
    raise
finally:
    reset_request_id(token)
    response.headers["x-request-id"] = request_id
    logger.info("HTTP request finished: status=%s elapsed=%.3fs path=%s",
response.status_code, time.perf_counter() - started, request.url.path)
    return response

    async def _handle(file: UploadFile, response_format: str = "json") ->
Response:
    metrics = _metrics
    if metrics is not None:
        metrics.request_start()
    if not is_model_loaded() or metrics is None or _pipeline_semaphore is
None:
        if metrics is not None:
            metrics.request_failed_before_acquire("handler")
            raise HTTPException(status_code=503, detail="Models are not
loaded")
        semaphore = _pipeline_semaphore
        acquired = False
        pipeline_task: asyncio.Task | None = None

        def release_after_pipeline_done(_: asyncio.Future) -> None:
            metrics.request_finished()
            semaphore.release()

        try:
            if QUEUE_TIMEOUT_S < 0:
                await semaphore.acquire()
            elif QUEUE_TIMEOUT_S == 0:
                if semaphore.locked():
                    metrics.request_rejected()
                    raise HTTPException(
                        status_code=429,
                        detail="No free inference slots. Try again later.",
                        headers={"Retry-After":
queue_retry_after(QUEUE_TIMEOUT_S)},
                    )
                await semaphore.acquire()
            else:
                await
                asyncio.wait_for(semaphore.acquire(),
timeout=QUEUE_TIMEOUT_S)
            except asyncio.TimeoutError as exc:
                metrics.request_rejected()
                raise HTTPException(
                    status_code=503,
                    detail="No free inference slots. Try again later.",
                    headers={"Retry-After": queue_retry_after(QUEUE_TIMEOUT_S)},
                ) from exc
            except asyncio.CancelledError:
                metrics.request_rejected(count_failure=False)
                raise
            acquired = True
            metrics.request_acquired()

            body = await file.read()
            started = time.perf_counter()

            def sync_transcribe() -> dict:
                return transcribe_sync(body, metrics=metrics,
slot_acquired=True)

```

```

        pipeline_task
    asyncio.create_task(asyncio.to_thread(sync_transcribe))
    pipeline_task.add_done_callback(release_after_pipeline_done)
    try:
        if INFER_TIMEOUT_S > 0:
            result = await asyncio.wait_for(
                asyncio.shield(pipeline_task), timeout=INFER_TIMEOUT_S)
            else:
                result = await asyncio.shield(pipeline_task)
    except asyncio.TimeoutError as exc:
        metrics.request_errored("timeout")
        raise HTTPException(status_code=504, detail=f"Inference timeout:
{INFER_TIMEOUT_S}s") from exc
    except asyncio.CancelledError:
        logger.info("Request cancelled by client: file=%s",
file.filename)
        raise

    metrics.request_done(
        audio_duration_s=result["performance"]["audio_duration_s"],
        elapsed_s=time.perf_counter() - started,
    )
    if LOG_TRANSCRIPTS:
        logger.info("Transcription result: chars=%d text=%s",
len(result["text"]), short_text(result["text"]))
    return format_result(
        text=result["text"],
        duration=result["performance"]["audio_duration_s"],
        segments=result.get("segments"),
        speakers=result.get("speakers"),
        diarization=result.get("diarization"),
        overlap_transcripts=result.get("overlap_transcripts"),
        performance=result.get("performance"),
        response_format=response_format,
    )

@app.post("/v1/audio/transcriptions")
async def v1_audio_transcriptions(
    file: UploadFile = File(...),
    response_format: str = Form("json"),
) -> Response:
    return await _handle(file, response_format)

@app.post("/audio/transcriptions")
async def audio_transcriptions(
    file: UploadFile = File(...),
    response_format: str = Form("json"),
) -> Response:
    return await _handle(file, response_format)

@app.get("/health")
async def health() -> dict:
    snapshot = _metrics.snapshot() if _metrics else {}
    return {
        "status": "ok" if is_model_loaded() else "loading",
        "model": MODEL_NAME,
        "diarization": runtime_info(),
        "pool_size": INFER_POOL_SIZE,
        "slots_free": max(0, INFER_POOL_SIZE -
snapshot.get("requests_active", 0)),

```

```

"queue_timeout_s": QUEUE_TIMEOUT_S,
"requests": {
    "total": snapshot.get("requests_total", 0),
    "ok": snapshot.get("requests_ok", 0),
    "failed": snapshot.get("requests_failed", 0),
    "failed_by_stage": snapshot.get("requests_failed_by_stage",
{}),
    },
}

```